

Team Assignment: Business Analytics — Decision Support or Agentic Automation

1. Overview

Submission Deadline: September 20

Each team of 5 students must identify a **real business problem or opportunity** that can be addressed using business analytics, and then build a working solution. Before building anything, your team must **argue whether the decision can or cannot be automated**. This argument determines which path you take:

- **Path A — Automation is possible:** Build an **agentic automation** using CrewAI or n8n that makes/executes the decision (e.g., credit card approval).
- **Path B — Automation is not possible/appropriate:** Build a **classical decision support application** that helps a human make a better decision, but does not decide for them (e.g., hire or not).

Both paths are equally valid. What matters is the quality of your reasoning and the quality of your build.

2. Learning Objectives

By the end of this assignment, you should be able to:

1. Identify and scope a real, non-trivial business problem suitable for analytics.
 2. Distinguish between decisions that are safe/appropriate to automate versus those that require human judgment, and justify that distinction with evidence.
 3. Design and build either:
 - a decision support system (data → analysis → recommendation, human decides), or
 - an agentic automation pipeline (data → analysis → decision → action, system decides/acts).
 4. Communicate technical work to a non-technical business audience.
-

3. Step-by-Step Instructions

Step 1: Find a Real Problem or Opportunity

- The problem must be **real** — from an actual organization (a local business, your university, a startup, an NGO, a family business, a publicly documented case) — not a hypothetical you invent from thin air.
- Examples of domains: hiring, credit/loan approval, inventory reordering, customer churn, dynamic pricing, fraud detection, supplier selection, marketing lead scoring, admissions, insurance claims triage, customer support ticket routing.
- You must be able to describe: *who* faces this problem, *how often* the decision is made, *what data* is available or obtainable, and *what happens today* (current process).
- You may use **publicly available datasets** (e.g., Kaggle, UCI Machine Learning Repository, government open-data portals, World Bank, company-published datasets) to support your analysis, as long as the underlying problem is genuinely real and the data is relevant to it. Clearly cite the source of any dataset you use.

Step 2: Argue — Can This Decision Be Automated?

This is the most important intellectual step of the assignment. Your team must explicitly argue **for or against automation**, considering:

Factor	Questions to ask
Reversibility	If the automated decision is wrong, how costly/reversible is it?
Data quality & availability	Is there enough clean, relevant, unbiased data to train/drive automated logic?
Legal & ethical risk	Are there regulations (e.g., anti-discrimination law), fairness concerns, or liability issues?
Stakes & frequency	Is this a high-volume, low-stakes decision (good automation candidate) or low-volume, high-stakes (needs human judgment)?
Explainability	Can the logic be justified to the affected person/regulator if challenged?
Human judgment/context	Does the decision require tacit knowledge, negotiation, or empathy that's hard to encode?

You must write this argument out formally (see Deliverables). Simply asserting "automation is possible" or "not possible" without reasoning against these factors will not receive credit.

Step 3a: If Automatable → Build an Agentic Automation

- Use **CrewAI** (Python-based multi-agent framework) or **n8n** (visual workflow automation tool) to build a working pipeline that:
 1. Ingests or simulates realistic input data (e.g., an applicant's credit profile).
 2. Applies rules, ML models, and/or LLM reasoning to reach a decision.
 3. Takes an action or produces an output autonomously (e.g., approve/reject + notify).
- The system should run end-to-end with at least 5-10 test cases showing different outcomes.
- Include guardrails: what happens on edge cases, low-confidence outputs, or errors? Where, if anywhere, does a human still get a checkpoint (human-in-the-loop)?

Step 3b: If Not Automatable → Build a Classical Decision Support Application

- Build an application using **Streamlit** (Python) that:
 1. Takes in relevant data about the case.
 2. Analyzes it (scoring models, comparisons, visualizations, statistical/ML analysis).
 3. Presents a **clear recommendation with supporting evidence** to a human decision-maker.
 4. Leaves the final decision to the human — the tool informs, it does not decide.
- Example: a "hire or not" scorecard that weighs candidate attributes against role requirements and shows the hiring manager a ranked comparison, but the manager makes the call.

Step 4: Test and Validate

- Run your system/application against multiple realistic scenarios/test cases.
- Document what worked, what didn't, and any surprising results.

Step 5: Present and Submit

Each team presents their work (format and duration to be confirmed by instructor) and submits a written report.

4. Deliverables

1. **Problem Brief (1 page)** — the real problem, organization/context, current process, why it matters.

2. **Automation Argument (1–2 pages)** — structured argument for/against automation using the factors in Step 2, with a clear conclusion.
 3. **Working Build:**
 - If automated: CrewAI/n8n project files or exported workflow + screen recording/demo of it running.
 - If decision support: the application/dashboard/model file + screen recording/demo.
 4. **Test Results** — evidence from at least 5–10 test cases/scenarios with outcomes.
 5. **Team Presentation** — summarizing problem, argument, build, and results.
 6. **Individual Reflection (optional/instructor's discretion)** — each member's specific contribution.
-

5. Suggested Timeline

Final submission is due September 20. Work backward from that date using a schedule like this:

Milestone	Suggested timing (before deadline)
Form teams, identify 2–3 candidate problems, pick one	~6 weeks before
Complete automation argument; get instructor sign-off on direction (A or B)	~5 weeks before
Build prototype (agent pipeline or Streamlit decision support app)	~4–2 weeks before
Test with realistic scenarios, refine	~2 weeks before
Prepare presentation, finalize report, submit	By September 20

(Adjust to your actual course calendar and start date.)

6. Evaluation Criteria (Suggested Rubric)

Criterion	Weight
Reality and relevance of the problem chosen	15%
Rigor of the automation-vs-decision-support argument	25%
Technical quality and functionality of the build	30%
Quality of testing/validation	15%
Clarity of presentation and documentation	15%

7. Tools Reference

- **CrewAI** — Python framework for building multi-agent systems where agents have roles, tools, and can hand off tasks to each other. Good for decision + action pipelines.
- **n8n** — visual, node-based workflow automation tool; good if your team prefers low-code and wants to connect to real APIs/services (email, Slack, databases, spreadsheets) easily.
- **Streamlit** — Python framework for building interactive web apps quickly; this is the required tool for decision support applications in this assignment. It lets you turn a data analysis/model into an interactive tool (upload data, adjust inputs, see recommendations/visualizations) without needing front-end development skills.

8. Academic Integrity Note

Using AI tools (including Claude, ChatGPT, GitHub Copilot, etc.) to help you code, research, or draft is permitted and encouraged, but your team must be able to explain and defend every design choice and every line of your automation argument in the presentation/Q&A.